

Resumo: Code Smells e Refatoração em Elixir

Lucas Vegi - (UFV)

MARCOS H. DE SOUZA ALMEIDA

1. O Problema e a Motivação

A maioria da literatura sobre *Code Smells* e Refatoração, como o livro clássico de Martin Fowler, é focada em Orientação a Objetos (Java, C++, etc.).

- **A Lacuna:** Desenvolvedores de linguagens funcionais como Elixir muitas vezes tentam aplicar regras de OO que não fazem sentido no paradigma funcional, ou ignoram problemas específicos que só existem nesse paradigma.
- **O Objetivo:** Criar um catálogo de problemas (smells) e soluções (refatorações) nativos e específicos para Elixir.

2. O que são "Smells" no contexto Elixir?

Lucas explica que *Code Smells* são estruturas no código que indicam que algo pode estar errado (design ruim), dificultando a manutenção. Em Elixir, eles se dividem em:

- **Tradicionais:** Smells clássicos que ainda se aplicam (ex: "Código Duplicado", "Nomes Ruins").
- **Específicos de Elixir:** Problemas que nascem de recursos únicos da linguagem (ex: uso incorreto de Macros, Pipe Operator ou Pattern Matching).

3. Exemplos de Smells Catalogados

O pesquisador apresenta alguns dos 35 smells identificados em sua pesquisa. Exemplos notáveis incluem:

- **Complex else clauses in `with`:**
 - O que é: O comando "`with`" é ótimo para "caminho feliz", mas quando você coloca muita lógica complexa no bloco "`else`" para tratar erros diferentes, o código fica ilegível.
 - Solução: Quebrar em funções menores ou usar *Fallbacks Controller* (em Phoenix).
- **Dynamic Atom Creation:**
 - O que é: Criar atoms (constantes) baseados em input de usuário (ex: "`String.to_atom(input)`").
 - Risco: Em Elixir, atoms não são coletados pelo Garbage Collector. Se um atacante enviar milhões de strings diferentes, ele pode estourar a memória do servidor (Ataque DoS).
- **Unnecessary Macros:**

- O que é: Usar metaprogramação (macros) para resolver problemas simples que poderiam ser resolvidos com funções normais.
- Problema: Aumenta a complexidade de leitura e debug.
- **Pipe Obsession:**
 - O que é: Tentar forçar tudo a caber dentro de um operador pipe `|>`, criando cadeias longas demais ou ilegíveis.

4. Ferramentas e Detecção

- **Credo:** A ferramenta padrão de análise estática (linter) para Elixir.
- **Limitação:** O Lucas aponta que o Credo, por padrão, detecta apenas uma parte pequena dos smells teóricos. A pesquisa dele visa expandir essas regras para cobrir problemas de design mais profundos, não apenas estilo.